

Como implementar segurança no código

Ponto de atenção — exemplos de código escritos para Spring Boot 3.x

Todos os trechos de `SecurityConfig.java`, `KeyAuthFilter.java` e da classe principal nesta documentação foram escritos e validados no Spring Boot 3.5, que usa o Spring Security 6.

O Spring Boot 4.1 traz o Spring Security 7, que removeu APIs utilizadas nestes exemplos — entre elas o `AntPathRequestMatcher`, usado aqui através do `antMatcher()`. Além disso, diversas classes de autoconfiguração foram reorganizadas em pacotes diferentes. Copiar este código em um projeto no Boot 4 resultará em erro de compilação.

Os exemplos serão atualizados após a primeira migração real para o Boot 4, quando as alterações puderem ser validadas em um projeto funcionando. Até lá, quem estiver migrando deve consultar o time de DevOps antes de adaptar estas configurações.

Equalizar versão do Spring

Atenção — a linha 3.x não recebe mais correções de segurança. O Spring Boot 3.5 encerrou o suporte open source em 30/06/2026, e a 3.5.16 foi a última versão publicada no Maven Central. Não haverá novas correções para nenhum branch 3.x.

Por isso, projetos que forem passar por atualização devem migrar para o Spring Boot 4.1.

pom.xml

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>x</version>
  <relativePath/>
</parent>
```

Utilizar a versão 4.1 mais recente disponível (verificar no Maven Central antes de fixar).

Incluir SonarQube na aplicação

pom.xml

```
<properties>
  ...
  <sonar.qualitygate.wait>true</sonar.qualitygate.wait>
  <sonar.dependencyCheck.htmlReportPath>./target/dependency-check-report.html</sonar.dependencyCheck.htmlReportPath>
  <sonar.dependencyCheck.jsonReportPath>./target/dependency-check-report.json</sonar.dependencyCheck.jsonReportPath>
  <sonar.dependencyCheck.summarize>true</sonar.dependencyCheck.summarize>
  <sonar.host.url>http://sonar.cooxupe.com.br:9000</sonar.host.url>
</properties>
```

Incluir Dependency Check na aplicação

pom.xml

```
<properties>
  ...
  <dependency-check-maven.version>x</dependency-check-maven.version>
  ...
</properties>
```

Adicionar o plugin abaixo do "dependency-check-maven". Antes de fechar a tag (verificar se há versão do plugin mais atualizada).

```
    <plugin>
      <groupId>org.owasp</groupId>
      <artifactId>dependency-check-maven</artifactId>
      <version>x</version>
      <configuration>
        <formats>
          <format>html</format>
          <format>json</format>
        </formats>
      </configuration>
    </plugin>
```

Configurações proibidas no pom.xml

O bloco `<configuration>` do plugin deve conter apenas os `<formats>`. As tags abaixo são bloqueadas pela pipeline e reprovam o job `dependency-scanning`:

- `<failBuildOnCVSS>` — o limiar é definido pela pipeline (valor 0), não pelo projeto
- `<suppressionFile>` — supressão local não é permitida

O motivo é que a configuração do POM tem precedência sobre os parâmetros da linha de comando. Uma dessas tags anularia silenciosamente a verificação de segurança, e a pipeline passaria com vulnerabilidades presentes.

Supressões devem ser solicitadas ao time de Segurança, no repositório `ci-cd/seguranca` (arquivo `Suppressions/dependency-check-global.xml`). O pedido precisa justificar por que a vulnerabilidade não se aplica ao contexto da aplicação ou por que não há correção disponível.

Adicionar certificado Digital SSL da Cooxupé

Criar uma pasta "keystore" no diretório "src/main/resources" para armazenar o certificado SSL no formato *.P12: api-certificate.p12.

application.properties

```
# =====
#  Configuracao SSL Certificate
# =====
server.ssl.key-store-type=PKCS12
server.ssl.key-store=classpath:keystore/api-certificate.p12
server.ssl.key-store-password=${API_CERT_PWD}
server.ssl.enabled=true
```

Caso a API seja acionada através do banco de dados, necessário realizar os seguintes ajustes:

- Criar ACL para o servidor de integração destino com a porta mínima e máxima da API.
- Criar a Oracle Wallet no banco de dados em uso.
- Ajustar a PROCEDURE e PACKAGE ???

Para testar localmente, utilizar da seguinte forma: https://{nomedamaquina}.cooxupe.com.br:{porta}

Adicionar API-KEY

Utilizar esse recurso para as APIs internas.

Os exemplos desta seção são para Spring Boot 3.x. Ver ponto de atenção no início do documento.

Cenário para adicionar API-KEY na API

Ajustar o nome da variável conforme a nomenclatura da API. Há dependência desse passo com o [módulo Spring Security](#).

application.properties

```
# =====
#  Api Key Config
# =====
http.auth-token-header-name=x-api-key
http.auth-token=${MODOK_API_KEY}
http.auth-token-ebs-list-ip= ${MODOK_AD_KEY}
```

Adicionar no arquivo "SecurityConfig.java".

```
public class SecurityConfig {

    @Value("${http.auth-token-header-name}")
    private String principalRequestHeader;

    @Value("${http.auth-token}")
    private String principalRequestValue;

    @Bean
    @Order
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        KeyAuthFilter filter = new KeyAuthFilter(principalRequestHeader);

        filter.setAuthenticationManager(new AuthenticationManager() {

            @Override
            public Authentication authenticate(Authentication authentication) throws AuthenticationException {
                String principal = (String) authentication.getPrincipal();

                if (principal.equals(verificarIps(principal))){
                    authentication.setAuthenticated(true);
                    return authentication;
                }

                if (!principalRequestValue.equals(principal)) {
                    authentication.setAuthenticated(false);

                    throw new BadCredentialsException("The API key was not found or not the expected value.");
                }

                authentication.setAuthenticated(true);
                return authentication;
            }
        });

        http.csrf(csrf -> csrf.disable())
            .httpBasic(b-> b.disable())
            .formLogin(f-> f.disable())
            .logout(l-> l.disable())
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(authorize -> authorize
                .requestMatchers(
                    antMatcher("/actuator"),
                    antMatcher("/actuator/**"),
                    antMatcher("/v3/api-docs/**"),
                    antMatcher("/swagger-ui.html"),
                    antMatcher("/swagger-ui/**")
                ).permitAll()

                .anyRequest().authenticated()
            ).addFilter(filter);

        http.requiresChannel(
            req -> req.requestMatchers(r -> r.getHeader("X-Forwarded-Proto") != null).requiresSecure();

            return http.build();
        }
    }
}
```

Cenário para adicionar API-KEY com lista na API

Utilizar esse recurso quando a API deve confiar na origem que a chama, por exemplo:

1. EBS_API_KEY: utilizado quando o EBS aciona alguma API.
2. PBI_API_KEY: utilizado quando o Power BI aciona alguma API.

Ajustar o nome das variáveis abaixo, conforme necessidade.

application.properties

```
# =====
#  Api Key Config
# =====

...

http.auth-token-ebs-list-ip= ${EBS_API_KEY}
```

Adicionar no arquivo "SecurityConfig.java".

```
import java.util.List;

public class SecurityConfig {
```

```
...

@Value("${http.auth-token-ebs-list-ip}")
private List<String> ebsIps;

...

private String verificarIps(String ebsApiKey) {
    return ebsIps.stream().filter(f -> f.equals(ebsApiKey)).map(ff -> ff).findAny().orElse("XXXX");
}

}
```

Cenário para aplicação onde é utilizada uma API para autenticar

application.properties

```
# =====
#  Api Key Config EBS
# =====

AUTH_EBS_URL=${AUTH_EBS_URL}
AUTH_EBS_MODULE=${AUTH_EBS_MODULE}
AUTH_EBS_API_KEY=${AUTH_EBS_API_KEY}
```

SecurityConfig.java

Adaptar as rotas conforme a aplicação.

```
public class SecurityConfig {
    @Bean
    SecurityFilterChain configure(HttpSecurity http) throws Exception {

        http.authorizeHttpRequests(authorize -> authorize
            .requestMatchers(
                antMatcher("/webjars/**"),
                antMatcher("/css/**"),
                antMatcher("/images/**"),
                antMatcher("/js/**"),
                antMatcher("/"),
                antMatcher("/home"),
                antMatcher("/expired"),
                antMatcher("/actuator"),
                antMatcher("/actuator/**"),
                antMatcher("/v3/api-docs/**"),
                antMatcher("/swagger-ui.html"),
                antMatcher("/swagger-ui/**"),
                antMatcher("/error/**")

            ).permitAll()

            .anyRequest().authenticated()
        ).formLogin(formLogin -> formLogin
            .loginPage("/login")
            .defaultSuccessUrl("/listar", true)
            .failureUrl("/login-error")
            .permitAll()
        ).logout(logout -> logout
            .logoutSuccessUrl("/login")
            .deleteCookies("JSESSIONID")
        ).exceptionHandling(ex -> ex
            .accessDeniedPage("/acesso-negado")
        ).rememberMe(
            withDefaults()
        ).sessionManagement(session -> session
            .maximumSessions(1)
            .expiredUrl("/expired")
            .maxSessionsPreventsLogin(false)
            .sessionRegistry(sessionRegistry())
        ).sessionManagement(session -> session
            .sessionFixation()
            .newSession()
            .sessionAuthenticationStrategy(sessionAuthStrategy())

        );

        return http.build();
    }

    ...

}
```

Criar uma classe "Service" para tratar a conexão de login STATEFULL através da API de autenticação.

Adicionar JWT na API

Utilizar esse recurso para as APIs externas.

```
Modelo a definir.
```

Módulo Spring Security

```
Adicionar o módulo Spring Security.

Criar um novo pacote para as novas classes do Spring Security

Criar as classes KeyAuthFilter.java e SecurityConfig.java
```

Observação: seguir com a configuração da API-KEY, conforme orientado no menu [Adicionar API-KEY](#)

Remoção do usuário default

Exemplo para Spring Boot 3.x. No Boot 4 as classes de autoconfiguração foram reorganizadas em outros pacotes, então o import desta classe deve mudar. Validar na primeira migração.

Adicionar o Exclude na classe principal:

```
@SpringBootApplication(exclude = {UserDetailsServiceAutoConfiguration.class })
```

Maiores orientações [nesse link](#).

Bibliotecas desatualizadas

Para verificar vulnerabilidades no projeto, fazer o CLEAN, INSTALL e BUILD com os parâmetros abaixo:

```
dependency-check:check
```

```
surefire-report:report
```

Atualizar as libs identificadas.